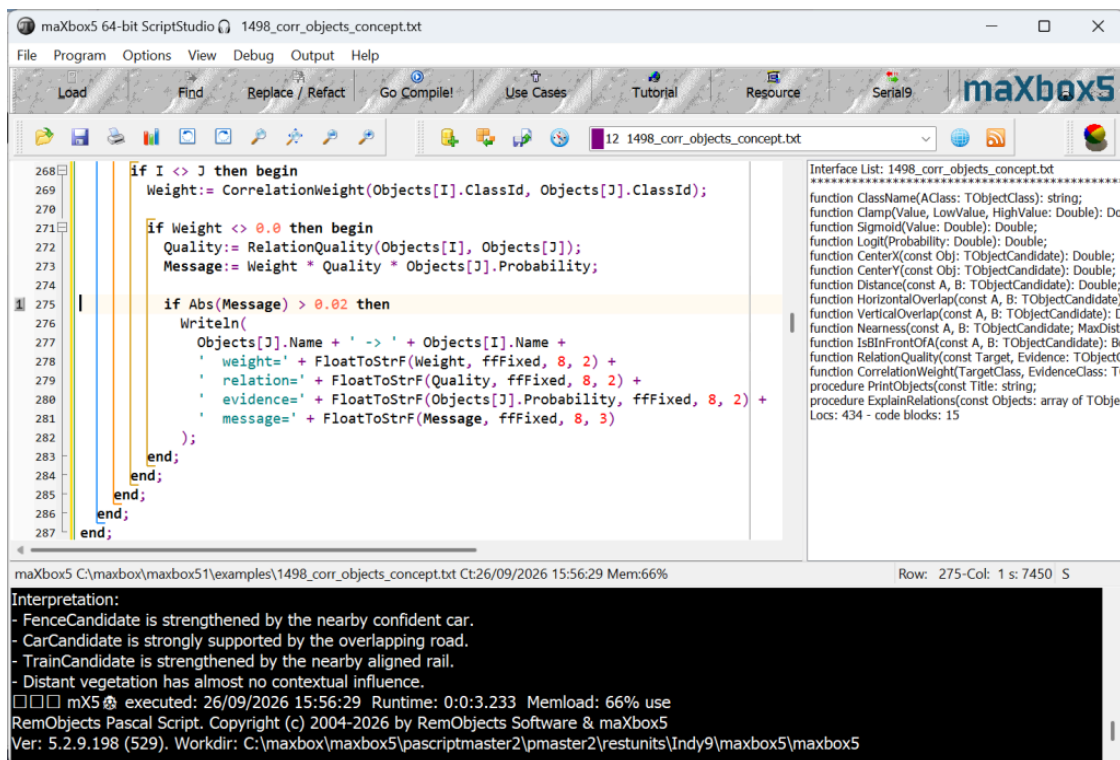




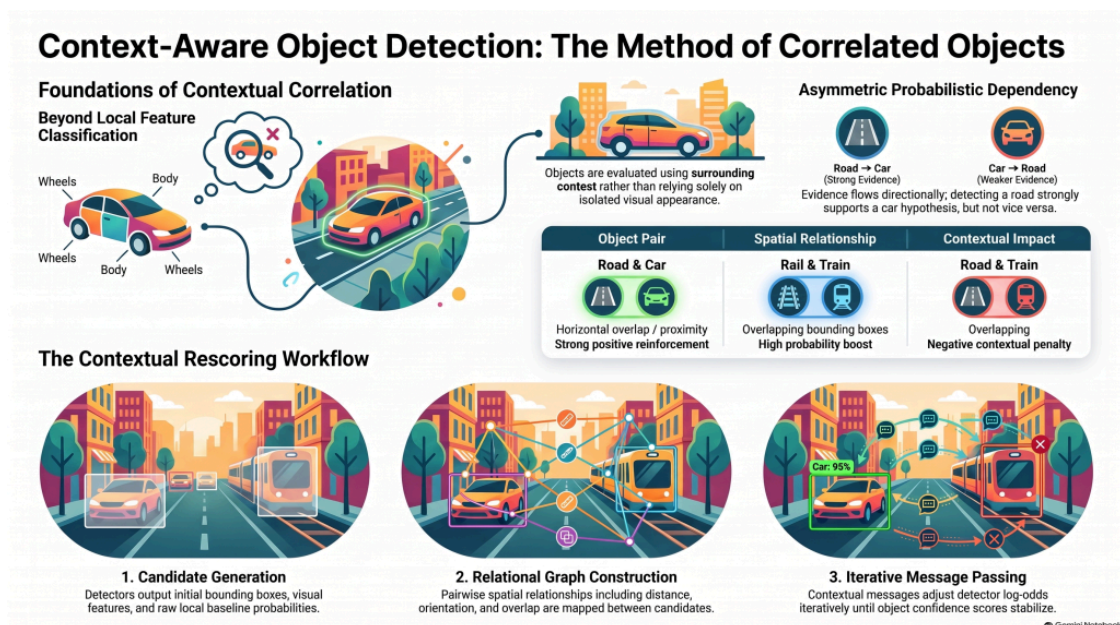
Thirty Years Alone

What happens when a state forgets to plan for human beings? A woman spent 30 years in solitary confinement.

Read the full story



from Tutor 198



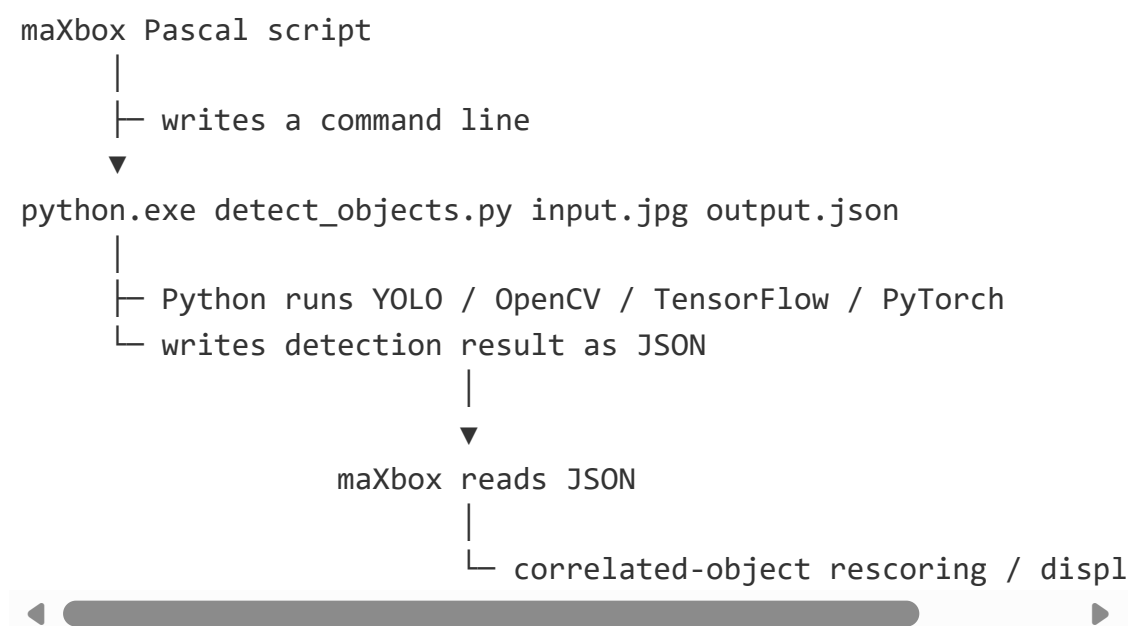
Integration with YOLO model and Python

In addition to Tutorial 198 and 198_1 will be 198_2

The most reliable way is to run Python **out of process** from maXbox, exchange files—preferably JSON—and let maXbox consume the result. This avoids the Python DLL bitness/version constraints of embedded Python, which are especially relevant because older maXbox/Python4Delphi setups are often 32-bit while current object-detection Python stacks are normally 64-bit. maXbox documentation specifically describes external-shell execution as the practical route for 64-bit Python.[\[softwareschule\]](#)[\[maxbox4.wordpress\]](#)

Recommended architecture

Use this pipeline:



This separation is robust because Python owns its own virtual environment and packages, while maXbox focuses on orchestration, GUI, rules, and your correlated-object algorithm.

The external command can be invoked with `ShellExecute`, `ShellExecute3`, `ExecuteShell`, or a command-output helper such as `getDosOutput`, depending on the maXbox version and registered units. The maXbox function

list documents `ShellExecute`, `ShellExecute2`, and `ShellExecute3`; the Python4Delphi examples also describe retrieving external Python command output into `maxbox`.[\[softwareschule\]](#)[\[softwareschule\]](#)

Python environment setup

Install a current 64-bit Python, then create a dedicated environment. In a Windows terminal:

```
py -3.12 -m venv C:\vision-env
C:\vision-env\Scripts\activate
python -m pip install --upgrade pip
pip install ultralytics opencv-python
```

For a first test, create this Python program as `detect_objects.py`. It uses Ultralytics YOLO and writes a portable JSON file containing labels, confidences, and bounding boxes.

```
import argparse
import json
from pathlib import Path

from ultralytics import YOLO

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--image", required=True)
    parser.add_argument("--output", required=True)
    parser.add_argument("--model", default="yolo11n.pt")
    parser.add_argument("--conf", type=float, default=0.25)
    args = parser.parse_args()

    image_path = Path(args.image)
    output_path = Path(args.output)

    if not image_path.exists():
        raise FileNotFoundError(f"Image not found: {image_path}")

    model = YOLO(args.model)
    results = model.predict(
        source=str(image_path),
        conf=args.conf,
        verbose=False
    )
```

```

result = results[0]
detections = []

for box in result.bboxes:
    class_id = int(box.cls[0].item())
    confidence = float(box.conf[0].item())
    x1, y1, x2, y2 = [float(v) for v in box.xyxy[0].tolist()

    detections.append(
        {
            "class_id": class_id,
            "class_name": result.names[class_id],
            "confidence": round(confidence, 6),
            "x": round(x1, 2),
            "y": round(y1, 2),
            "width": round(x2 - x1, 2),
            "height": round(y2 - y1, 2)
        }
    )

payload = {
    "image": str(image_path),
    "model": args.model,
    "confidence_threshold": args.conf,
    "object_count": len(detections),
    "objects": detections
}

output_path.parent.mkdir(parents=True, exist_ok=True)
output_path.write_text(
    json.dumps(payload, indent=2),
    encoding="utf-8"
)

print(f"Detected {len(detections)} objects")
print(f"JSON result written to: {output_path}")

if __name__ == "__main__":
    main()

```

Run it once directly before calling it from maXbox:

```

C:\vision-env\Scripts\python.exe detect_objects.py ^
--image C:\vision\input.jpg ^
--output C:\vision\result.json ^

```

A typical result file looks like this:

```
{
  "image": "C:\\vision\\input.jpg",
  "model": "yolo11n.pt",
  "confidence_threshold": 0.25,
  "object_count": 2,
  "objects": [
    {
      "class_id": 2,
      "class_name": "car",
      "confidence": 0.932411,
      "x": 160.0,
      "y": 160.0,
      "width": 54.0,
      "height": 30.0
    },
    {
      "class_id": 0,
      "class_name": "person",
      "confidence": 0.821144,
      "x": 315.0,
      "y": 111.0,
      "width": 27.0,
      "height": 86.0
    }
  ]
}
```

The `python script.py` command-line pattern is the normal cross-platform method for launching Python scripts; Windows can also use the `py` launcher. [\[realpython\]](https://realpython.com/using-the-py-launcher/)

maXbox launcher script

This maXbox program starts the Python script, waits for completion by invoking it through `cmd.exe`, captures console text, and then displays the JSON file.

Adapt the three path constants to your installation. Keep the explicit path to `python.exe`; do not depend on whichever Python happens to be on `PATH`.

```
program PythonObjectDetectionLauncher;
```

```
const
```

```
    PYTHON_EXE = 'C:\\vision-env\\Scripts\\python.exe';
```

```
DETECTOR_SCRIPT = 'C:\vision\detect_objects.py';
INPUT_IMAGE = 'C:\vision\input.jpg';
RESULT_JSON = 'C:\vision\result.json';

function Quote(const S: string): string;
begin
    Result := '"' + S + '"';
end;

function BuildCommand: string;
begin
    Result :=
        Quote(PYTHON_EXE) + ' ' +
        Quote(DETECTOR_SCRIPT) + ' ' +
        '--image ' + Quote(INPUT_IMAGE) + ' ' +
        '--output ' + Quote(RESULT_JSON) + ' ' +
        '--conf 0.25';
end;

function ReadTextFile(const FileName: string): string;
var
    F: TextFile;
    Line: string;
begin
    Result := '';

    AssignFile(F, FileName);
    Reset(F);

    try
        while not Eof(F) do begin
            ReadLn(F, Line);
            Result := Result + Line + #13#10;
        end;
    finally
        CloseFile(F);
    end;
end;

var
    CommandLine: string;
    ConsoleOutput: string;
    JsonText: string;

begin
    Writeln('Python / maXbox object-detection bridge');
```

```
Writeln('=====');

if not FileExists(PYTHON_EXE) then begin
  Writeln('ERROR: Python executable not found:');
  Writeln(PYTHON_EXE);
  Exit;
end;

if not FileExists(DETECTOR_SCRIPT) then begin
  Writeln('ERROR: Python detector script not found:');
  Writeln(DETECTOR_SCRIPT);
  Exit;
end;

if not FileExists(INPUT_IMAGE) then begin
  Writeln('ERROR: Input image not found:');
  Writeln(INPUT_IMAGE);
  Exit;
end;

CommandLine := BuildCommand;

Writeln('');
Writeln('Starting Python:');
Writeln(CommandLine);
Writeln('');

{ getDosOutput runs the command synchronously and returns std
  If your maXbox installation does not register getDosOutput,
  use ExecuteShell / ShellExecute3 and poll for RESULT_JSON i
  ConsoleOutput := getDosOutput('cmd.exe /C ' + CommandLine);

Writeln('Python console output:');
Writeln('-----');
Writeln(ConsoleOutput);

if not FileExists(RESULT_JSON) then begin
  Writeln('ERROR: No JSON output file was created. ');
  Writeln('Check the Python console output above. ');
  Exit;
end;

JsonText := ReadTextFile(RESULT_JSON);

Writeln('');
Writeln('JSON detection result:');
```

```

Writeln('-----');
Writeln(JsonText);

Writeln('');
Writeln('Done.');
```

`getDosOutput` is convenient because it returns standard output to the maXbox console; the maXbox Python integration material describes this pattern for calling `py` with parameters and returning Python output. [\[softwareschule\]](#)

Feeding your correlated-object algorithm

Your prior correlated-object program needs each detection in the form:

```

Name
ClassId
LocalProbability
X, Y, W, H
```

The JSON output maps directly:

JSON field	Correlated-object field
class_name	Class mapping, e.g. "car" → ocCar
confidence	LocalProbability
x, y	X, Y
width, height	W, H

Add a class mapper in maXbox:

```

function MapClassName(const S: string): TObjectClass;
var
  T: string;
begin
  T := LowerCase(S);

  if T = 'car' then
    Result := ocCar
  else if T = 'train' then
    Result := ocTrain
  else if T = 'person' then
    Result := ocUnknown
```

```

else
    Result := ocUnknown;
.

```

For your railway/GEOINT use case, a stock COCO YOLO model is only a starting point: it detects common labels such as **car**, **truck**, **bus**, and **train**, but generally not domain-specific labels like **rail**, **switch**, **catenary mast**, **signal**, **platform edge**, or **fence**. For those, train or fine-tune a custom YOLO model and change:

```
--model yolo11n.pt
```

to, for example:

```
--model C:\vision\models\railway_detector.pt
```

Then define the same class names in the Python dataset YAML and in the `maXbox MapClassName` function.

Embedded Python alternative

`maXbox` can also integrate Python in process through `Python4Delphi`. In that setup, a `maXbox` Pascal script loads a compatible Python DLL, initializes it, and invokes Python statements or files. The `maXbox/Python4Delphi` material describes functions such as `Py_Initialize`, `PyRun_SimpleString/PyRun_SimpleStringFlags`, and `ExecString`.
[\[maxbox4.wordpress\]](https://maxbox4.wordpress.com/2026/09/24/method-of-correlated-objects/)[\[softwareschule\]](https://www.softwareschule.de/)

Conceptually:

```

const
    PYDLLPATH = 'C:\Python37-32\';
    PYDLLNAME = 'python37.dll';

begin
    { Load Python DLL, initialize interpreter, then execute code.
    Import('Py_Initialize', PYDLLPATH + PYDLLNAME);
    Import('PyRun_SimpleString', PYDLLPATH + PYDLLNAME);

    Py_Initialize;
    PyRun_SimpleString(
        'print("Hello from embedded Python")' + #10 +
        'value = 2 + 2' + #10 +
        'print(value)'
    );
end;

```

However, I would **not** recommend this route for modern object detection unless you specifically need it:

- Embedded Python must match the maXbox process architecture—32-bit maXbox needs 32-bit Python and compatible compiled packages.
- Current PyTorch, TensorFlow, ONNX Runtime, CUDA bindings, and many OpenCV builds are much easier to install and maintain in an isolated 64-bit virtual environment.
- A model crash, native-library issue, or memory leak in embedded Python can affect the maXbox process directly.
- External JSON exchange makes inference reproducible and easy to debug.

The maXbox documentation explicitly notes the historical 32-bit limitation of embedded Python and recommends an external shell for 64-bit Python installations.[\[softwareschule\]](#)[\[maxbox4.wordpress\]](#)

Production improvements

For repeated analysis, use these refinements:

- Load the YOLO model once in a long-running Python worker instead of starting Python for every image.
- Use JSON Lines over standard input/output, local HTTP, ZeroMQ, or a named pipe for a persistent worker.
- Add **image_width**, **image_height**, model version, inference time, and coordinate-reference metadata to JSON.
- Keep raw detector confidence separate from the maXbox context-adjusted confidence.
- Write the raw detections and correlation rules to an audit log.
- Calibrate detector confidences before treating them as probabilities, especially if a context module can significantly adjust scores.
- Use a custom model for railway infrastructure rather than trying to infer specialist objects solely through correlation.

A useful next architecture would be: Python produces raw YOLO detections as JSON; maXbox runs your explicit, inspectable graph-based correlation algorithm; maXbox exports a second JSON file with **local_confidence**, **context_score**, **final_confidence**, and a list of supporting neighbour relations for every object.

Description with Import DLL

The script is ready. Here is a summary of what it does:

Three-Level Python4Delphi Integration

Level 1 – Raw DLL external declarations: Direct linkage to `Py_Initialize`, `PyRun_SimpleString`, and `Py_Exit` via the external `'FunctionName@DLLPath cdecl'` directive, exactly as shown in the maXbox Python4Delphi tutorials. A direct `pyRunSimpleString('print(...)')` call demonstrates this level.[\[softwareschule\]](#)

Level 2 – `Import()` function pointers: After `eng.loadDLL`, the script explicitly calls `Import('PyRun_SimpleString')`, `Import('Py_Initialize')`, `Import('Py_Exit')`, `Import('PyModule_GetDict')`, `Import('PyDict_GetItemString')`, `Import('PySys_SetArgv')`, `Import('Py_BuildValue')`, and `Import('PyImport_ImportModule')` to resolve typed function pointers — the same mechanism `TPythonEngine` uses internally, as described in the Python4Delphi source.[\[github\]](#)

Level 3 – `TPythonEngine` high-level API: Uses `eng.ExecString()` for Python statements and `eng.EvalStr()` for expressions, the standard maXbox pattern from [softwareschule.ch](#) examples.[\[softwareschule.code\]](#)

Object Detection Pipeline

1. Initialize: `TPythonEngine.Create(nil)`, set `pythonhome`, call `loadDLL` (which calls `Py_Initialize` internally)
2. Redirect output: `io.StringIO` buffer captures Python `print()` output back to maXbox console
3. Load model: `ExecString` runs the `PY_SETUP` const that imports `ImageAI`, creates `ObjectDetection`, calls `setModelTypeAsTinyYOLOv3()`, `setModelPath()`, and `loadModel()` — following the maXbox Starter75 object detection tutorial[\[es.slideshare\]](#)
4. Pass image frame: Pascal string path is embedded in the Python `run_detection()` call via `ExecString`
5. Parse bounding boxes: Two methods are demonstrated:
 - Helper method: Single `EvalStr` call returns a pipe-delimited `"name|prob|x1,y1,x2,y2"` string, split in Pascal
 - Direct method: Individual `EvalStr` calls access `_detections[i]` `["box_points"][0..3]` directly
6. Display: Summary table with object name, confidence percentage, and box coordinates (x1, y1, x2, y2, width, height)

7. Open output: `openFile()` displays the annotated image with bounding boxes drawn by ImageAI

Key Configuration

Adjust `PYDLL_PATH` and `PYHOME` constants to match your Python installation. The bitness (32-bit or 64-bit) must match your maXbox build. You need `pip install imageai tensorflow opencv-python numpy` and the [yolo-tiny.h5 model file](#) placed in `.\models\`.

Advertisements

Occasionally, some of your visitors may see an advertisement here, as well as a [Privacy & Cookies banner](#) at the bottom of the page. You can hide ads completely by upgrading to one of our paid plans.

UPGRADE NOW

DISMISS MESSAGE

Posted in [Code](#), [Geo Intelligence](#), [Geomap](#), [Trains](#)

One response to “Method of correlated objects”



maxkleiner

September 26, 2026 [Edit](#)

Can you give me a programm for maXbox with source code for an algorithm for correlated objects?

★ Like

[Reply](#)

Leave a comment
